03. 기본 명령어

> **이 단원이 시험의 절반입니다.** 2차 1과목 48문항 중 대부분이 명령어 문제입니다. 명령어 자체보다 **옵션 문자**를 정확히 구분하는 것이 핵심입니다.

---

1. 디렉터리 이동·확인

```
pwd             # 현재 디렉터리 절대경로 출력
cd /etc         # 절대경로 이동
cd ..           # 상위 디렉터리
cd ~            # 홈 디렉터리 (cd 만 입력해도 동일)
cd ~user1       # user1의 홈 디렉터리
cd -            # 직전 디렉터리로 복귀
```

기호	의미
`.`	현재 디렉터리
`..`	상위 디렉터리
`~`	현재 사용자 홈 디렉터리
`~user`	특정 사용자 홈 디렉터리
`-`	이전 작업 디렉터리
`/`	루트 디렉터리

---

2. ls — 목록 출력

```
ls -l           # 상세 정보(long)
ls -a           # 숨김 파일 포함(all, . 으로 시작하는 파일)
ls -A           # 숨김 포함하되 . 과 .. 제외
ls -d           # 디렉터리 자체 정보 (내용 아님)
ls -h           # 사람이 읽기 쉬운 단위(K, M, G)
ls -i           # inode 번호 출력
ls -R           # 하위 디렉터리까지 재귀적
ls -t           # 수정 시간 순 정렬
ls -S           # 파일 크기 순 정렬
ls -r           # 역순 정렬
ls -F           # 파일 유형 표시 (/ 디렉터리, * 실행, @ 링크)
ls -lrt         # 자주 쓰는 조합: 시간 역순 상세
```

`ls -l` 출력 해석 (시험 단골)

```
-rw-r--r--.  1 root root  1234 Sep 23 10:00 test.txt
 | └──┬──┘ └┬┘ └┬┘    |   |     |      |         |         |
 |    |     |   |     |   |     |      |         |         └ 파일명
 |    |     |   |     |   |     |      |         └ 최종 수정 시간
 |    |     |   |     |   |     |      └ 파일 크기(바이트)
 |    |     |   |     |   |     └ 그룹
 |    |     |   |     |   └ 소유자
 |    |     |   |     └ 하드링크 수
 |    |     |   └ other 권한
 |    |     └ group 권한
 |    └ user(소유자) 권한
 └ 파일 종류
```

첫 문자(파일 종류)

기호	의미
`-`	일반 파일
`d`	디렉터리
`l`	**심볼릭 링크**
`b`	블록 장치 (디스크)
`c`	문자 장치 (터미널, 프린터)
`p`	파이프(FIFO)
`s`	소켓

---

3. 파일·디렉터리 생성·삭제·복사·이동

```
touch file.txt          # 빈 파일 생성 / 기존 파일의 시간 갱신
touch -t 202612120900 f # 시간 지정
mkdir dir1              # 디렉터리 생성
mkdir -p a/b/c          # 중간 경로까지 한 번에 생성 (parents)
mkdir -m 755 dir        # 권한 지정 생성
rmdir dir1              # 빈 디렉터리만 삭제
rmdir -p a/b/c          # 상위까지 삭제

rm file                 # 파일 삭제
rm -r dir               # 디렉터리 재귀 삭제 (recursive)
rm -f file              # 강제 삭제, 확인 없음 (force)
rm -i file              # 삭제 전 확인 (interactive)
rm -rf dir              # 재귀 강제 삭제 ← 가장 위험

cp a b                  # 복사
cp -r dir1 dir2         # 디렉터리 복사 (필수 옵션)
cp -i                   # 덮어쓰기 전 확인
cp -p                   # 속성(소유자 · 권한 · 시간) 보존 (preserve)
cp -a                   # 아카이브 = -dpR (원본 그대로)
cp -u                   # 원본이 더 최신일 때만 복사 (update)
cp -f                   # 강제
cp -v                   # 진행 상황 출력 (verbose)

mv a b                  # 이동 또는 이름 변경
```

```
mv -i                  # 덮어쓰기 확인
mv -f                  # 강제
mv -u                  # 최신일 때만
mv -b                  # 덮어쓸 파일 백업
```

출제 포인트

- **디렉터리 복사는 `-r` 필수**, `cp dir1 dir2` 만 하면 오류
- `mv` 는 같은 파일시스템 내에서는 이름만 바뀌므로 inode가 유지됨
- `rmdir` 은 **빈 디렉터리만** 삭제 가능 → 내용이 있으면 `rm -r`

---

4. 파일 내용 보기

```
cat file               # 전체 출력
cat -n file            # 행 번호 표시
cat -b file            # 빈 줄 제외하고 번호 표시
cat f1 f2 > f3         # 연결(concatenate)
tac file               # 역순 출력 (cat의 반대)

more file              # 페이지 단위 (앞으로만)
less file              # 페이지 단위 (앞/뒤 모두, more보다 강력)

head file              # 앞 10줄 (기본)
head -n 5 file         # 앞 5줄
head -5 file           # 동일
tail file              # 뒤 10줄
tail -n 20 file        # 뒤 20줄
tail -f /var/log/messages  # 실시간 추적 (follow) ★로그 확인 단골

nl file                # 행 번호 붙여 출력
wc file                # 행/단어/바이트 수
wc -l file             # 행(line) 수만
wc -w file             # 단어(word) 수
wc -c file             # 바이트(character) 수
od file                # 8진수 등으로 덤프
file test.txt          # 파일 종류 판별
strings bin            # 바이너리에서 문자열 추출
```

출제 포인트

- `tail -f` = 로그 실시간 모니터링 (가장 많이 나옴)
- `wc -l` = 행 수 (`-l` 이 line)
- `less` vs `more`: less는 **뒤로 스크롤 가능**

---

5. 검색 — find (매우 중요)

```
find /home -name "*.txt"          # 이름으로 검색
find / -iname "abc*"              # 대소문자 무시
find /tmp -type f                 # 파일만 (d=디렉터리, l=링크)
find / -user user1                # 소유자로 검색
find / -group admin               # 그룹으로 검색
find / -perm 644                  # 권한이 정확히 644
find / -perm -4000                # SetUID가 설정된 파일 ★보안 점검 단골
find / -size +10M                 # 10MB 초과
find / -size -100k                # 100KB 미만
find / -empty                     # 빈 파일/디렉터리
find / -mtime -7                  # 7일 이내 수정된 파일
find / -mtime +30                 # 30일 이전 수정
find / -atime -1                  # 1일 이내 접근
find / -newer ref.txt             # ref.txt보다 최신
find / -maxdepth 2 -name "*.conf"  # 탐색 깊이 제한
find /tmp -name "*.tmp" -exec rm -f {} \;   # 찾아서 실행 ★
find /tmp -name "*.log" -delete    # 찾아서 삭제
find . -name "a*" -o -name "b*"    # OR 조건
find . ! -name "*.txt"            # 부정(NOT)
```

시간 옵션 3종 구분 (자주 출제)

옵션	기준
`-atime`	**A**ccess — 마지막 접근(읽기) 시각
`-mtime`	**M**odify — 마지막 내용 수정 시각
`-ctime`	**C**hange — 마지막 속성(inode) 변경 시각

`+n` = n일보다 이전, `-n` = n일 이내, `n` = 정확히 n일 전

다른 검색 명령

```
which ls          # PATH 내 실행 파일 경로
whereis ls        # 실행파일 + 소스 + man 페이지 경로
locate passwd     # DB 기반 빠른 검색 (updatedb로 DB 갱신)
updatedb          # locate용 DB 갱신
type cd           # 명령어 유형(내장/외부/별칭) 확인
```

명령	특징
`find`	실시간 디스크 탐색 → 느리지만 정확
`locate`	**DB 검색** → 매우 빠르지만 DB가 오래되면 부정확
`which`	PATH 환경변수 내에서만 검색
`whereis`	바이너리·소스·매뉴얼 위치

6. 문자열 검색 — grep

```
grep "root" /etc/passwd
grep -i "ROOT" file        # 대소문자 무시 (ignore case)
grep -v "root" file        # 일치하지 않는 행 (invert) ★
grep -n "root" file        # 행 번호 표시
grep -c "root" file        # 일치하는 행 개수만 (count)
grep -l "root" *           # 일치하는 파일명만 출력
grep -r "root" /etc        # 하위 디렉터리 재귀 검색
grep -w "root" file        # 단어 단위로 정확히 일치
grep -E "a|b" file         # 확장 정규표현식 (= egrep)
grep -F "lit" file         # 고정 문자열 (= fgrep)
grep -A 3 "err" file       # 일치 행 뒤 3줄 함께
grep -B 3 "err" file       # 일치 행 앞 3줄 함께
```

정규표현식 기초

기호	의미
`^`	행의 시작
`$`	행의 끝
`.`	임의의 한 문자
`*`	앞 문자 0회 이상 반복
`[abc]`	a 또는 b 또는 c
`[^abc]`	a, b, c 가 아닌 문자
`[a-z]`	범위
`\`	이스케이프
`+`	(확장) 1회 이상 반복
`?`	(확장) 0 또는 1회
`|`	(확장) OR

예: `grep "^root" /etc/passwd` → root로 시작하는 행 `grep "bash$" /etc/passwd` → bash로 끝나는 행

---

7. 압축과 아카이브 (시험 단골 ★★★)

압축 명령

명령	확장자	압축 해제	특징
`compress`	`.Z`	`uncompress`	구형
`gzip`	`.gz`	`gunzip`, `gzip -d`	가장 일반적
`bzip2`	`.bz2`	`bunzip2`, `bzip2 -d`	gzip보다 압축률 높음
`xz`	`.xz`	`unxz`, `xz -d`	압축률 최고
`zip`	`.zip`	`unzip`	윈도우 호환, **여러 파일 묶기 가능**

핵심 차이: `gzip`, `bzip2`, `xz`는 **파일 하나만** 압축(묶기 불가). 여러 파일을 묶으려면 `tar`가 필요합니다. `zip`은 묶기+압축을 동시에 합니다.

```
gzip file          # file → file.gz (원본 삭제)
gzip -d file.gz    # 압축 해제
gzip -l file.gz    # 압축 정보 확인 (list)
gzip -r dir        # 디렉터리 내 파일 각각 압축
gzip -9 file       # 최대 압축률 (1=빠름, 9=최대)
zcat file.gz       # 압축 해제 없이 내용 보기
```

tar — 아카이브 (반드시 암기)

```
tar cvf backup.tar dir/        # 묶기 (create)
tar xvf backup.tar             # 풀기 (extract)
tar tvf backup.tar             # 목록 보기 (list) ★
tar rvf backup.tar newfile     # 기존 아카이브에 추가 (append)
tar uvf backup.tar file        # 갱신된 것만 추가 (update)

tar zcvf backup.tar.gz dir/    # gzip 압축과 함께 묶기 ★★
tar zxvf backup.tar.gz         # gzip 압축 풀기 ★★
tar jcvf backup.tar.bz2 dir/   # bzip2 압축
tar jxvf backup.tar.bz2        # bzip2 해제
tar Jcvf backup.tar.xz dir/    # xz 압축 (대문자 J)
tar Jxvf backup.tar.xz         # xz 해제

tar xvf b.tar -C /opt          # 지정 디렉터리에 풀기
tar cvf b.tar --exclude=*.log dir/   # 특정 파일 제외
```

옵션	의미
`c`	create — 새 아카이브 생성
`x`	extract — 풀기
`t`	list — 내용 목록 보기
`r`	append — 뒤에 추가
`u`	update — 갱신된 파일만 추가
`v`	verbose — 과정 출력
`f`	file — 아카이브 파일명 지정 (**거의 항상 필요**)
`z`	**gzip** 연동 (`.tar.gz`, `.tgz`)
`j`	**bzip2** 연동 (`.tar.bz2`)
`J`	**xz** 연동 (`.tar.xz`)
`p`	권한 보존
`C`	대상 디렉터리 지정

⚠️ **최다 출제 함정**: `z`(gzip)와 `j`(bzip2)와 `J`(xz) 구분. 그리고 `t`가 "목록 보기"라는 점.

---

8. 링크 — ln

```
ln origin hardlink       # 하드 링크
ln -s origin symlink     # 심볼릭(소프트) 링크 ★
```

구분	하드 링크	심볼릭 링크
inode	**원본과 동일**	별도 inode
원본 삭제 시	**데이터 유지** (링크로 접근 가능)	**깨짐**(dangling)
다른 파일시스템	**불가**	가능
디렉터리 대상	**불가**	가능
`ls -l` 표시	`-` (일반 파일)	`l` + `-> 원본`
링크 수 카운트	증가	증가하지 않음
윈도우 비유	-	바로가기

> 출제 포인트: **하드 링크는 디렉터리와 다른 파일시스템에 만들 수 없다.**

---

9. 리다이렉션과 파이프

```
command > file        # 표준출력 → 파일 (덮어쓰기)
command >> file       # 표준출력 → 파일 (이어쓰기)
command < file        # 파일 → 표준입력
command 2> file       # 표준에러 → 파일
command > file 2>&1   # 표준출력 + 표준에러 → 같은 파일 ★
command &> file       # 위와 동일(bash)
command 2> /dev/null  # 에러 메시지 버리기
command1 | command2   # 파이프 — 앞 출력을 뒤 입력으로
```

표준 스트림 번호 (암기)

번호	이름	기본 대상
0	표준 입력 (stdin)	키보드
1	표준 출력 (stdout)	모니터
2	표준 에러 (stderr)	모니터

tee — 화면 출력과 파일 저장 동시에

```
ls -l | tee out.txt       # 화면에도 보이고 파일에도 저장
ls -l | tee -a out.txt    # 이어쓰기
```

---

10. 텍스트 처리 필터

```
sort file             # 오름차순 정렬
sort -r file          # 내림차순 (reverse)
sort -n file          # 숫자 기준 정렬 (numeric)
sort -k 3 file        # 3번째 필드 기준
```

```
sort -u file          # 중복 제거 정렬
sort -t: -k3 -n /etc/passwd   # 구분자 : , 3번째 필드 숫자 정렬

uniq file             # 연속된 중복 행 제거 (정렬 후 사용)
uniq -c file          # 중복 횟수와 함께
uniq -d file          # 중복된 행만
uniq -u file          # 중복 없는 행만

cut -d: -f1 /etc/passwd     # : 구분, 1번 필드 추출 ★
cut -d: -f1,3 /etc/passwd   # 1,3번 필드
cut -c1-5 file              # 1~5번째 문자

tr 'a-z' 'A-Z' < file       # 소문자 → 대문자 변환
tr -d '\r' < file           # 특정 문자 삭제
tr -s ' ' < file            # 연속 문자 하나로 압축

paste f1 f2           # 두 파일을 행 단위로 붙이기
join f1 f2            # 공통 필드로 결합
split -l 100 file     # 100행 단위로 분할
diff f1 f2            # 두 파일 차이 비교
cmp f1 f2             # 바이너리 비교
comm f1 f2            # 정렬된 두 파일 공통/차이

sed 's/old/new/' file       # 첫 번째 매칭만 치환
sed 's/old/new/g' file      # 전체 치환 (global) ★
sed -i 's/old/new/g' file   # 파일 직접 수정 (in-place)
sed -n '5,10p' file         # 5~10행만 출력
sed '3d' file               # 3행 삭제

awk '{print $1}' file           # 1번째 필드 출력
awk -F: '{print $1, $3}' passwd  # 구분자 지정
awk '{print NR, $0}' file       # 행 번호와 전체 행
awk '$3 > 500 {print $1}' file   # 조건부 출력
awk 'END {print NR}' file       # 총 행 수
```

awk 내장 변수

변수	의미
`$0`	전체 행
`$1`, `$2` …	n번째 필드
`NR`	현재 행 번호 (Number of Records)
`NF`	현재 행의 필드 개수 (Number of Fields)
`FS`	입력 필드 구분자
`OFS`	출력 필드 구분자

---

11. 기타 유용한 명령

```
date                  # 현재 날짜 · 시간
date +%Y-%m-%d        # 형식 지정
cal                   # 달력
cal 12 2026           # 특정 월 달력
uptime                # 가동 시간 + 부하 평균
who                   # 현재 로그인 사용자
```

```
w                       # 로그인 사용자 + 작업 내용
whoami                  # 현재 사용자명
id                      # UID, GID, 그룹 정보
last                    # 최근 로그인 기록
users                   # 로그인 사용자명 목록
hostname                # 호스트명
uname -a                # 시스템 전체 정보
free -h                 # 메모리 사용량
df -h                   # 디스크 여유 공간 ★
du -sh dir              # 디렉터리 사용량 (summary, human) ★
echo "hello"            # 출력
printf "%s\n" "hi"      # 형식 출력
clear                   # 화면 지우기
history                 # 명령 이력
alias ll='ls -l'        # 별칭 설정
unalias ll              # 별칭 해제
man ls                  # 매뉴얼
man 5 passwd            # 섹션 지정 매뉴얼
info ls                 # info 문서
ls --help               # 간단한 도움말
```

`df` vs `du` (혼동 주의)

- `df` = **d**isk **f**ree → **파일시스템 단위** 여유 공간
- `du` = **d**isk **u**sage → **디렉터리/파일 단위** 사용량

man 페이지 섹션 번호

섹션	내용
1	사용자 명령어
2	시스템 콜
3	라이브러리 함수
4	장치 파일 (`/dev`)
5	**설정 파일 형식** (`/etc/passwd` 등)
6	게임
7	기타
8	**관리자 명령어**
9	커널 루틴

---

12. 마무리 체크 문제

1. 디렉터리를 복사하려면 필수 옵션? → `-r`
2. `tar` 로 `.tar.bz2` 를 푸는 명령? → `tar jxvf`
3. `tar` 로 아카이브 **내용만 확인**하는 옵션? → `t`
4. 로그 파일을 실시간으로 보는 명령? → `tail -f`
5. 일치하지 않는 행만 출력하는 grep 옵션? → `-v`
6. SetUID가 설정된 파일을 찾는 명령? → `find / -perm -4000`
7. 하드 링크를 디렉터리에 만들 수 있는가? → **아니오**

8. 원본을 삭제해도 데이터가 살아있는 링크? → **하드 링크**
9. 표준 에러의 파일 디스크립터 번호? → **2**
10. 파일시스템 여유 공간을 보는 명령? → `df -h`
11. `/etc/passwd`에서 사용자명만 뽑는 명령? → `cut -d: -f1 /etc/passwd`
12. 파일 내 문자열을 전부 치환? → `sed -i 's/old/new/g' file`
13. `-atime`, `-mtime`, `-ctime` 각각의 기준? → 접근 / 내용 수정 / 속성 변경
14. `locate`가 `find`보다 빠른 이유? → **미리 만들어둔 DB를 검색**하기 때문
15. 설정 파일 형식을 설명하는 man 섹션 번호? → **5**